Customer Care Process Automation

Two bots replacing two manual back-office jobs, with a rule about when not to decide.

Taiwo Alabi · MSc Artificial Intelligence, National College of Ireland

Contents

1. The problem

Hundreds of customer complaints sorted by hand every morning, on a Nigerian mobile network. A person reads each one and decides which back-end team it belongs to: SIM activation, network, refunds, payment and billing, or general.

Two things go wrong. People misroute tickets, because after the sixtieth complaint they all start to read the same. And the customer waits while their ticket finds the right desk, then waits again once it gets there.

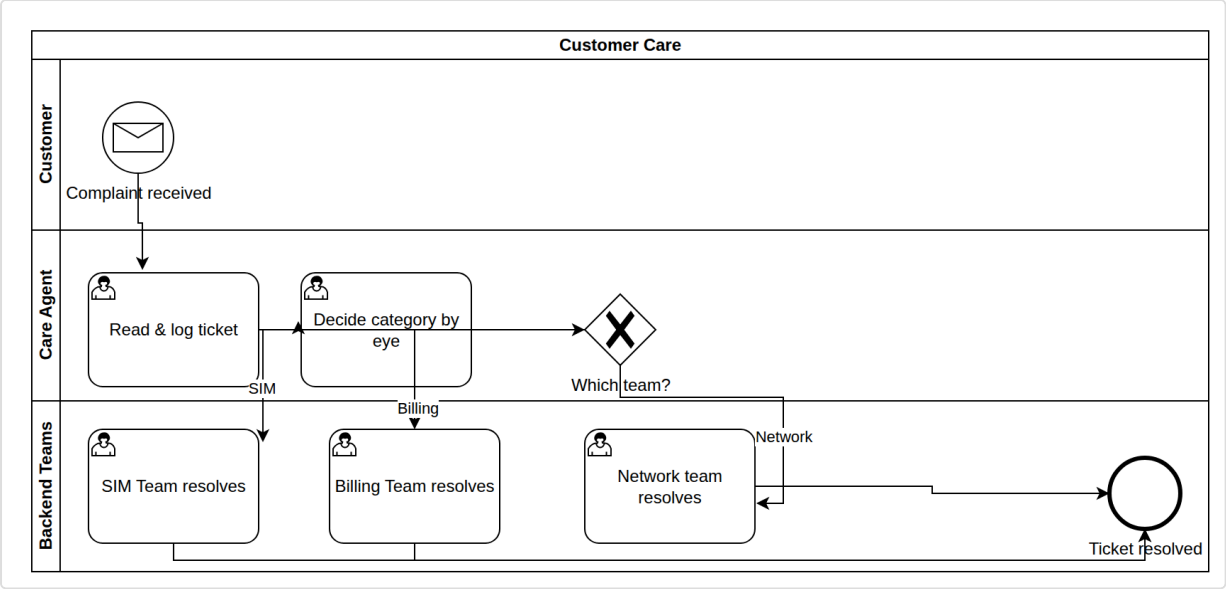
A misroute is not a small error. It costs the customer days, and the business a second handling pass on work it already paid for once.

2. Results

Measure	Result
Routing accuracy on held-out tickets	**95.2%** (40 of 42)
Labelled corpus	210 tickets, 42 per category
Categories	5
Confidence threshold for escalation	0.55
Manual processes replaced	2

The accuracy number is the least interesting thing here. What matters is what the agent does with the tickets it is **not** confident about, which is section 6.

3. The process, before

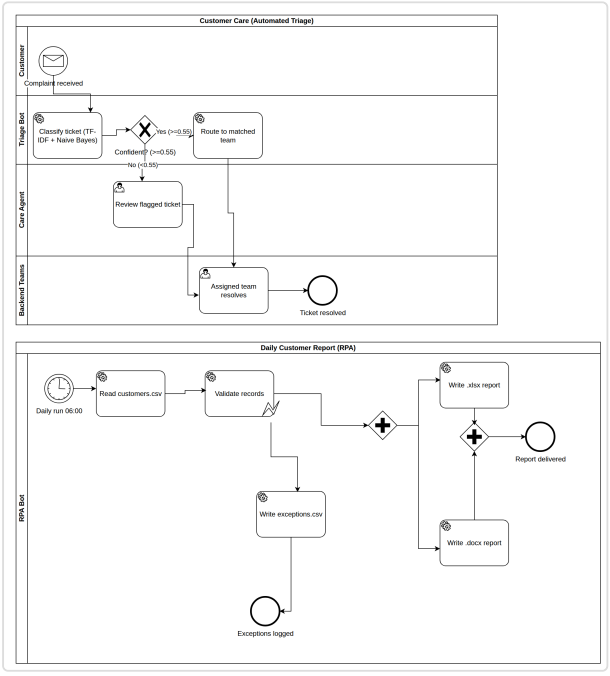


Every complaint read and routed by a person.

I modelled the existing process in BPMN 2.0 before writing any code. That order was deliberate: it meant the automation replaced a process I actually understood, rather than my assumption of one, and the model is what made the manual touchpoints explicit enough to decide which were worth automating.

---

4. The process, after



Two pools. Automated triage above, the daily customer report bot below.

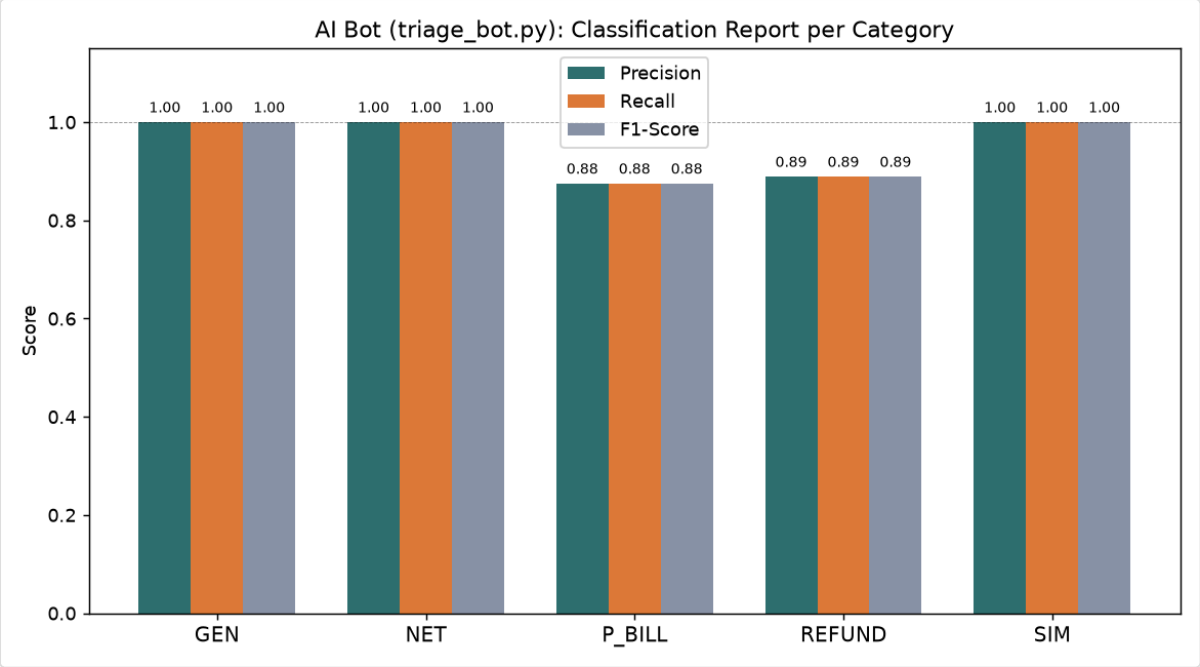
The redesign splits the work by the kind of decision each part requires. Field validation has known correct answers, so it goes to rules. Reading a complaint and inferring intent does not, so it goes to a classifier. The confidence gate sits between the classifier and the outcome.

---

5. How the triage agent works

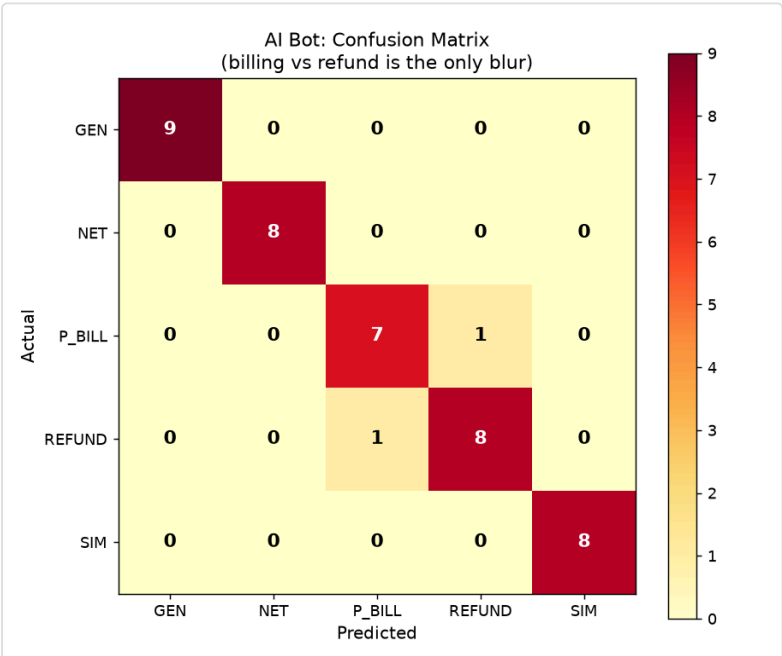
Ticket text is vectorised with TF-IDF and classified by multinomial Naive Bayes across the five routing categories.

Naive Bayes is a deliberate choice rather than a default. The corpus is 210 tickets, which is small, and Naive Bayes remains stable at that size where a heavier model would overfit. It is also inspectable: I can see which terms drive a routing decision, which matters when somebody asks why a ticket went where it did.



Per-class precision, recall and F1.

Reported per class rather than as a single figure, because a router that is strong on common categories and weak on rare ones is a different system from one that is evenly good, and an average hides that difference.

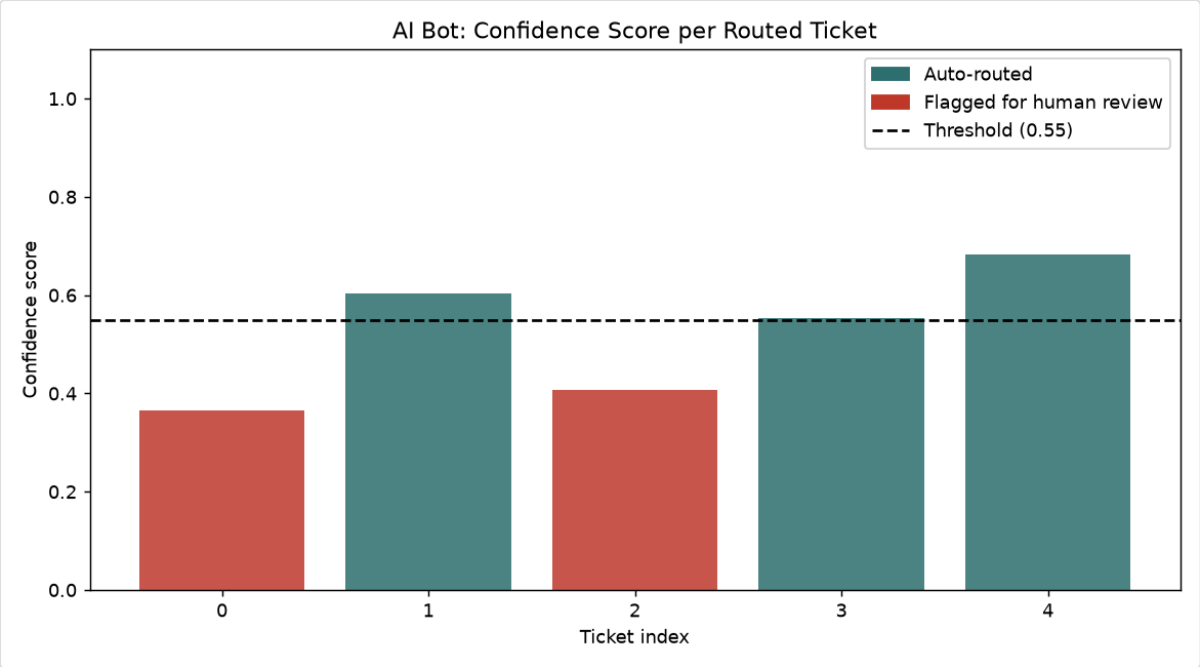


Where the classifier confuses one team with another.

The matrix shows where errors concentrate. Confusion between neighbouring categories is a routing problem worth fixing; errors scattered evenly would suggest the features themselves are too weak, and that is not the pattern.

---

6. The decision that matters: when not to decide



The escalation gate. Below 0.55, the ticket goes to a human.

A classifier will always return an answer. Asked to route a ticket it has never seen anything like, it will still pick the closest of the five categories and report a number. That is the failure mode worth designing against.

So the agent does not have to answer. Any ticket scoring below **0.55** confidence is flagged `needs_human_review` and escalated instead of routed.

The economics justify it. A needless escalation costs an agent about five minutes. A confident misroute costs the customer days. When one error is two orders of magnitude more expensive than the other, the correct system is one that escalates freely, and I would rather report a lower automation rate than a higher misroute rate.

---

7. The second bot: rule-based validation

The validator checking each record against its requirements.

The second automation produces the daily customer creation report: it reads the customer records, validates each field, writes the clean report to Excel and Word, and writes everything that failed validation to a separate exceptions file.

This half is deliberately **not** machine learning. These are hard business rules with known correct answers, so a classifier would introduce uncertainty where none needs to exist. Choosing not to use ML is as much a design decision as choosing to.

Outputs: `routed_tickets.csv`, `validation_exceptions.csv`, and the daily report in both `.xlsx` and `.docx`.

---

8. Honest limitations

- **The corpus is small and synthetic.** 210 generated tickets across five categories. Real complaints are messier: code-switching, abbreviations, several issues in one message. The generator was written to be realistic, but it is still a generator.
- **42 test tickets is a thin evaluation.** 95.2% means 40 of 42, so a single ticket moves the figure by more than two points. The pattern of errors is more informative than the headline number.
- **Five categories is a simplification.** A real network has more teams and less clean boundaries between them.
- **The confidence threshold was set by judgement, not swept.** 0.55 reflects the cost asymmetry described above, but a proper sweep against real escalation costs would be the honest way to fix it.

---

9. Running it

```
python gen_data.py          # build the labelled corpus
python triage_bot.py        # train, evaluate, route
python rpa_report_bot.py    # validate records, write the daily report
python plot_results.py      # regenerate every figure in this document
```

Seeded, so results reproduce on every run.

Stack: Python · scikit-learn (TF-IDF, Naive Bayes) · pandas · BPMN 2.0 · openpyxl · python-docx